Capstone Paper: AI-Powered Stock Market Tracking and Agent Training Application

(1) Description of the Project

This capstone project aimed to develop a comprehensive application for tracking the stock market and training artificial intelligence (AI) agents to identify patterns and make trading decisions. Recognizing the inherent complexity of the stock market, the application was designed with three key functionalities: real-time data acquisition, AI agent simulation and training, and back-testing capabilities to evaluate strategy performance. The final product integrates multiple AI trading agents (Simple Moving Average, RSI, RNN) along with visual representations of market data, agent performance, and back testing results. The overall goal was to create a platform for exploring the intersection of finance and artificial intelligence, while also gaining practical experience in application development and AI model training.

(2) Description of the Development Process

Deliverable 1 involved establishing a baseline AI agent and a simulated stock market. The baseline AI agent I found was a Simple Moving Average Strategy agent. The strategy works by dividing the sum of prices over N periods over N. The logic behind the equation is if the average is positive, it predicts the market is bullish. If the average is negative, it predicts the market is bearish. I also focused on figuring out which fundamental programs I needed to gather market data and host my code on a website. For hosting my code, I chose Streamlit due to it being both free and easily compatible with Python code. I had prior experience with Streamlit, making it an easy transition to include it in this project. If my code in GitHub is connected with the site, it can be opened anywhere on any device with just a link. The way I found to gather current market data is by utilizing a free API. Many free-market APIs are available, like Alpaca markets and Marketstack, but I chose Yahoo Finance (yfinance) because the documentation looked easy to understand and you can call the API very quickly throughout the day. Some difficulties I had **were** getting used to Yahoo Finance's functions, specifically the difference between yfinance.download() vs Ticker.history(). The difference between the two is download() can gather many different tickers at once, while history() gathers only one. For the future, I chose to stick with history() because it simplifies this project by keeping Simple Moving Average and future agents consistent with their data.

Deliverable 2 expanded upon this by trying to incorporate two additional agents (RSI and Simple RNN) and a basic back-testing feature. RSI, or Relative Strength Index, is a momentum oscillator that measures whether a stock is overbought or oversold. That means it compares the average of gains and losses over a time window, giving more of a symbol when to buy, hold, and sell. The calculation for RSI is 100 – (100/1+RS), RS (Relative Strength) being the average gain of up periods divided by the average loss of down periods. The best part about RSI is that it's very customizable for how risky it is. For example, you can set that if RSI > 70, that means the stock is overbought so it should sell. If RSI < 30, that means the stock is undersold so it should buy, and everything in between it will hold too. The creativity from this strategy comes from how you want to set those values to, like instead of 70:30, it could be 60:40 or 80:20. RNN was a ton more complicated, so I'll describe it more in Deliverable 3, when I actually got it working. Some difficulties I had were merging both yfinance and Streamlit together. Because I wanted to be close to current time market tracking, it required learning how often I can call the yfinance API and how to display it in

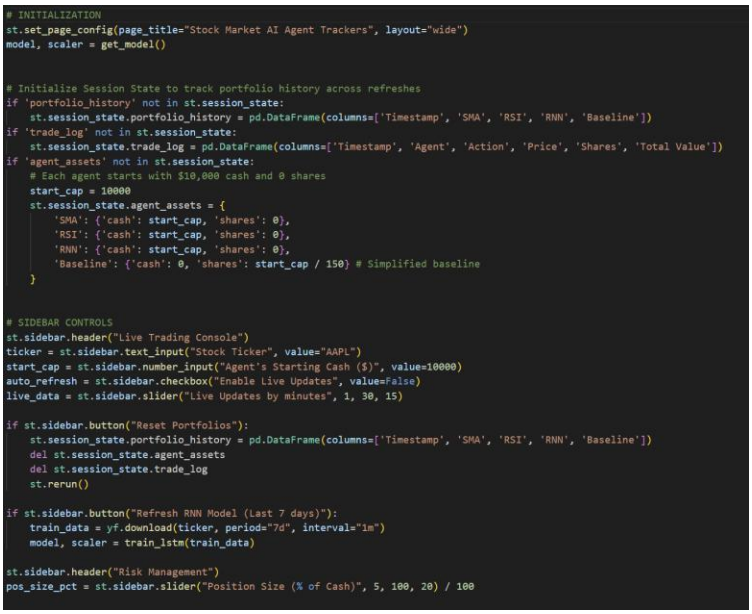
Streamlit. I thought using Streamlit's built in line_chart() function would work, but to display candles in my project I chose to add an extra Python library called plotly. This allows me to have way more customization in my graphs, like candles and multiple lines, that connected seamlessly to Streamlit. As I said before, I couldn't get RNN to work just yet due to it being more like an actual AI model instead of equations like SMA and RSI. Back testing was also a challenge because I needed every agent to be compatible with it. Back testing takes the 'signals' from the other agents and then figures out whether to buy, hold, or sell.

Lastly, Deliverable 3 finally had a working RNN model, live market tracking, and more customizability across all features. RNN, or Recurrent Neural Network, is a machine learning approach that tries to learn patterns in sequential price data. Unlike SMA or RSI, RNN learns from historical sequences where it checks the sequence of past prices to predict the future price or signal. It does this by maintaining a sort of memory that carries information from previous steps. There are many different RNN models, but what I chose to train in was LSTM (Long Short-Term Memory), which seems to be the most common trading. Making a model requires looking through a week's worth of market data to make it useable, so I added an option to overwrite an old model with a new model on the project. Other features I added were a stock portfolio to see how the agents were doing compared to a baseline, and a way to see current and previous signals for each agent. Some difficulties were getting this project published on Streamlit, requiring a requirements.txt to make sure the site was viewable anywhere. Another difficulty was making sure the site was getting live data. I knew it couldn't be perfectly live due to the lag of getting data from Yahoo but getting data every minute was needed for the agents. I tried using time.sleep(), but that caused issues with Streamlit not being useable during that minute. I fixed it by adding another Python library called streamlit-autorefresh, which allows the site to be useable while being updated.

(3) Final Deliverable & Functionality

Below are snippets of the code for the final version of this project:

Below are variables and Streamlit functions for designing the initial titles and graphs in the site. In CS_app.py



```
# INITIALIZATION
st.set_page_config(page_title="Stock Market AI Agent Trackers", layout="wide")
model, scaler = get_model()


# Initialize Session State to track portfolio history across refreshes
if 'portfolio_history' not in st.session_state:
    st.session_state.portfolio_history = pd.DataFrame(columns=['Timestamp', 'SMA', 'RSI', 'RNN', 'Baseline'])
if 'trade_log' not in st.session_state:
    st.session_state.trade_log = pd.DataFrame(columns=['Timestamp', 'Agent', 'Action', 'Price', 'Shares', 'Total Value'])
if 'agent_assets' not in st.session_state:
    # Each agent starts with $10,000 cash and 0 shares
    start_cap = 10000
    st.session_state.agent_assets = {
        'SMA': {'cash': start_cap, 'shares': 0},
        'RSI': {'cash': start_cap, 'shares': 0},
        'RNN': {'cash': start_cap, 'shares': 0},
        'Baseline': {'cash': 0, 'shares': start_cap / 150} # Simplified baseline
    }


# SIDEBAR CONTROLS
st.sidebar.header("Live Trading Console")
ticker = st.sidebar.text_input("Stock Ticker", value="AAPL")
start_cap = st.sidebar.number_input("Agent's Starting Cash ($)", value=10000)
auto_refresh = st.sidebar.checkbox("Enable Live Updates", value=False)
live_data = st.sidebar.slider("Live Updates by minutes", 1, 30, 15)

if st.sidebar.button("Reset Portfolios"):
    st.session_state.portfolio_history = pd.DataFrame(columns=['Timestamp', 'SMA', 'RSI', 'RNN', 'Baseline'])
    del st.session_state.agent_assets
    del st.session_state.trade_log
    st.rerun()

if st.sidebar.button("Refresh RNN Model (Last 7 days)"):
    train_data = yf.download(ticker, period="7d", interval="1m")
    model, scaler = train_lstm(train_data)

st.sidebar.header("Risk Management")
pos_size_pct = st.sidebar.slider("Position Size (% of Cash)", 5, 100, 20) / 100
```

Below fetches the data from the stock market, showing whether it's opened or closed. It also converts the market time to EST, gathers data for the RNN model, and gets the agents signals for their portfolios. In CS_app.py

```
# UI LAYOUT
st.title("Real-Time Agent Performance")

data = fetch_data(ticker)

# Checks if market is closed/open
t = yf.Ticker(ticker)
status = t.info.get('marketState')
# Common states: 'REGULAR', 'CLOSED', 'PRE', 'POST', 'PREPRE'


if not data.empty:
    # Flatten Multi-Index columns so the chart can find 'Open', 'Close', etc.
    if isinstance(data.columns, pd.MultiIndex):
        data.columns = data.columns.get_level_values(0)

    try:
        data.index = data.index.tz_convert('America/New_York')
    except TypeError:
        data.index = data.index.tz_localize('UTC').tz_convert('America/New_York')

    st.subheader(f"The market is currently in {status} trading hours.")

    current_p = data['Close'].iloc[-1].item()
    last_ts = data.index[-1]
    model, scaler = load_lstm()
    signals, rsi_val = get_signals(data, model, scaler)

    # Run logic update
    update_portfolios(current_p, signals, last_ts, pos_size_pct)
```

Below is to make sure Streamlit will refresh with the live data on the market with streamlit-autorefresh. In CS_app.py

```
# NEW AUTO-REFRESH LOGIC
if auto_refresh:
    # interval is in milliseconds (60000ms = 1 minute)
    # The 'key' ensures Streamlit tracks this specific timer
    st_autorefresh(interval=live_data*60000, key="trading_update_timer")
```

Below is the function for getting data during the day and loading the RNN model. In CS_agents.py

```
# FETCHING THE DATA
def fetch_data(symbol):
    df = yf.download(symbol, period="1d", interval="1m")
    return df

@st.cache_resource
def get_model():
    return load_lstm()
```

Below is the get_signals() function, getting the functions for SMA, RSI, and RNN. In CS_agents.py

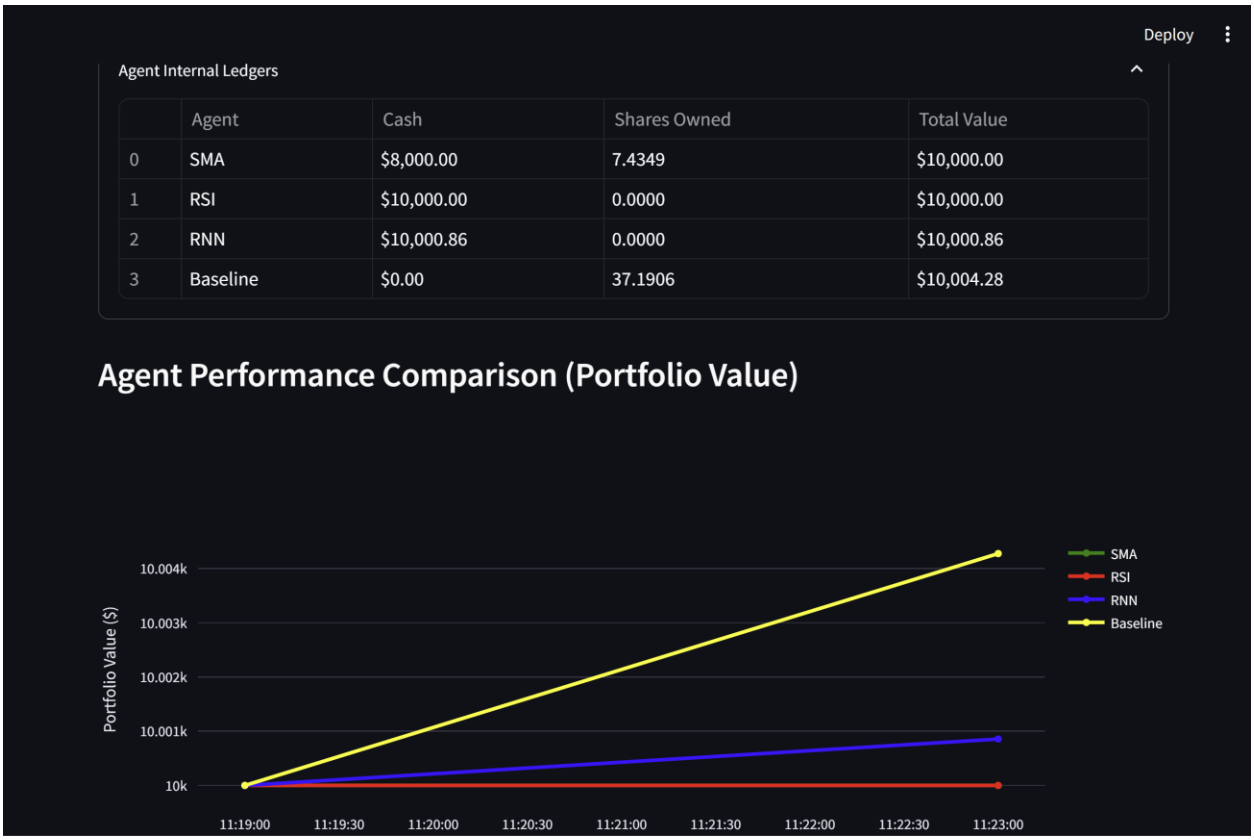
```
# GETTING SIGNALS FROM EACH AGENT
def get_signals(df, model, scaler):
    df = df.copy()

    if isinstance(df.columns, pd.MultiIndex):
        df.columns = df.columns.get_level_values(0)

    df = df.loc[:, ~df.columns.duplicated()]

    # SMA
    sma_fast = df['Close'].rolling(window=5).mean()
    sma_sig = 1 if float(df['Close'].iloc[-1]) > float(sma_fast.iloc[-1]) else -1

    # RSI
    delta = df['Close'].diff()
    gain = (delta.where(delta > 0, 0)).rolling(window=14).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=14).mean()

    rs = gain / loss.replace(0, float('nan'))
    rsi_val = float((100 - (100 / (1 + rs))).iloc[-1])

    rsi_sig = 1 if rsi_val < 40 else (-1 if rsi_val > 60 else 0)

    # RNN Logic
    lookback = 60

    if len(df) >= lookback and model is not None and scaler is not None:
        # 1. Get last 60 closes
        recent_data = df[['Close']].tail(lookback)

        # 2. Use TRAINED scaler
        scaled_data = scaler.transform(recent_data)

        # 3. Shape for LSTM
        input_data = scaled_data.reshape(1, lookback, 1)

        # 4. Predict (scaled output)
        prediction_scaled = model.predict(input_data, verbose=0)

        # 5. Convert back to real price
        prediction = scaler.inverse_transform(prediction_scaled)[0][0]

        current_price = float(df['Close'].iloc[-1])

        # 6. Signal
        rnn_sig = 1 if prediction > current_price else -1

    else:
        rnn_sig = 0

    return {'SMA': sma_sig, 'RSI': rsi_sig, 'RNN': rnn_sig}, rsi_val
```

Below is the function that updates the portfolios for each agent. In CS_agents.py

```python
# PORTFOLIO TRACKING
def update_portfolios(current_price, signals, timestamp, pos_size_pct):
    new_entry = {'Timestamp': timestamp}

    for agent in ['SMA', 'RSI', 'RNN']:
        asset = st.session_state.agent_assets[agent]
        sig = signals[agent]

        # BUY LOGIC: Only spend a percentage of current cash
        if sig == 1 and asset['cash'] > 10:  # Check if we have at least $10
            cash_to_spend = asset['cash'] * pos_size_pct
            shares_to_buy = cash_to_spend / current_price
            asset['shares'] += shares_to_buy
            asset['cash'] -= cash_to_spend

            # Log the trade
            st.session_state.trade_log = pd.concat([st.session_state.trade_log, pd.DataFrame([{
                'Timestamp': timestamp, 'Agent': agent, 'Action': 'BUY',
                'Price': current_price, 'Shares': shares_to_buy, 'Total Value': asset['cash'] + (asset['shares'] * current_price)
            }])], ignore_index=True)

        # SELL LOGIC: Sell all shares (or modify to sell a percentage)
        elif sig == -1 and asset['shares'] > 0:
            cash_received = asset['shares'] * current_price
            asset['cash'] += cash_received

            # Log the trade
            st.session_state.trade_log = pd.concat([st.session_state.trade_log, pd.DataFrame([{
                'Timestamp': timestamp, 'Agent': agent, 'Action': 'SELL',
                'Price': current_price, 'Shares': asset['shares'], 'Total Value': asset['cash']
            }])], ignore_index=True)

            asset['shares'] = 0

        # Calculate Current Value for history
        new_entry[agent] = asset['cash'] + (asset['shares'] * current_price)


    # Update Baseline (Buy and Hold)
    if st.session_state.agent_assets['Baseline']['shares'] == (10000/150): # First run init
        st.session_state.agent_assets['Baseline']['shares'] = 10000 / current_price

    new_entry['Baseline'] = st.session_state.agent_assets['Baseline']['shares'] * current_price

    # Append to History
    st.session_state.portfolio_history = pd.concat([
        st.session_state.portfolio_history,
        pd.DataFrame([new_entry])
    ], ignore_index=True)
```

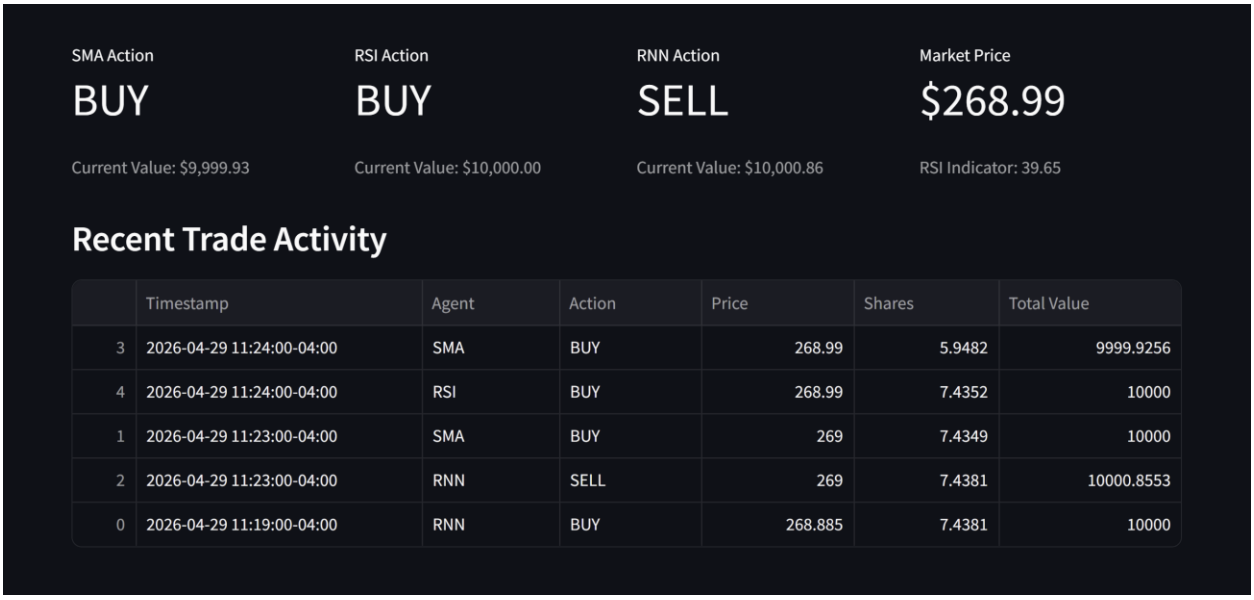
Below is the first look at the Streamlit site, showing live data, market activity, the changeable stock ticker, and changeable live updates. You can also edit the starting cash and % of cash each agent will trade and sell, and two buttons for reseting the portfolios and refresh the RNN model.

Scrolling down shows the cash, shares, and total value for each agent and baseline. It shows in both a table and graph for easy comparison.



	Agent	Cash	Shares Owned	Total Value
0	SMA	$8,000.00	7.4349	$10,000.00
1	RSI	$10,000.00	0.0000	$10,000.00
2	RNN	$10,000.86	0.0000	$10,000.86
3	Baseline	$0.00	37.1906	$10,004.28

At the bottom shows what action the agents are doing and have previously done.



	Timestamp	Agent	Action	Price	Shares	Total Value
3	2026-04-29 11:24:00-04:00	SMA	BUY	268.99	5.9482	9999.9256
4	2026-04-29 11:24:00-04:00	RSI	BUY	268.99	7.4352	10000
1	2026-04-29 11:23:00-04:00	SMA	BUY	269	7.4349	10000
2	2026-04-29 11:23:00-04:00	RNN	SELL	269	7.4381	10000.8553
0	2026-04-29 11:19:00-04:00	RNN	BUY	268.885	7.4381	10000

(4) Reflective Conclusion

Creating this Real-Time Market Tracker and AI Trader was a lot of fun for me, which tested my skills as a programmer and proved to be an incredibly valuable learning experience within finances. I gained significant practical skills in application development, AI model training, and financial data analysis. The development process allowed me to quickly research within a new field and create a blend of what I didn't know yet and what I knew. If I were to start this project again, I would prioritize more customization within the agents and the ability to trade within the site itself. Some things I would like to add in the future are realistic trading fees when buying and selling, improving the site by making it more presentable along with documentation, and extra trading strategies provided. This project gave me further interest in finance and AI.

(5) Important Links

Public GitHub: https://github.com/AndrewFagan04/Capstone_2026

Streamlit site: https://capstone2026-stockmarket-ai-tracker.streamlit.app/